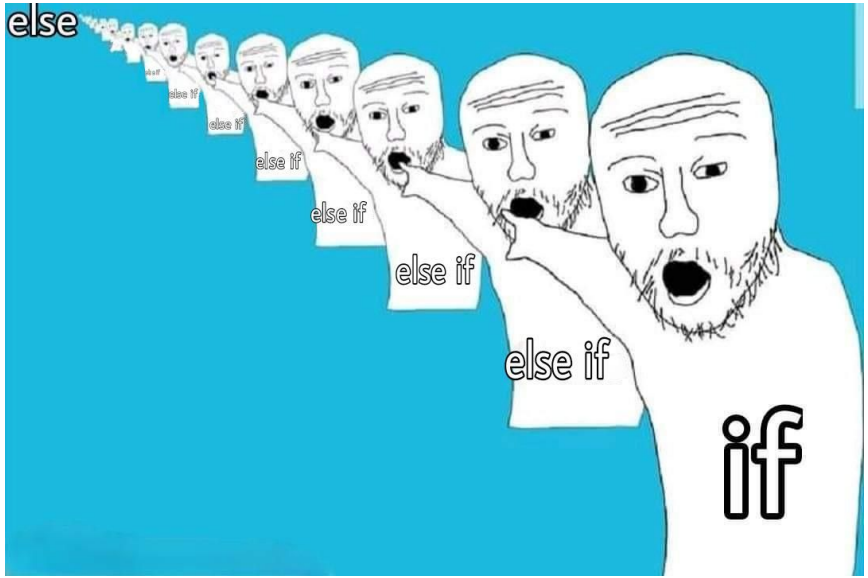
programando com python

etapa 2: what if... (lógica condicional)

e seus operadores lógicos e relacionais



- ➔ operadores relacionais
- ➔ operadores lógicos
- ➔ fluxo condicional
- ➔ condicionais if else elif
- ➔ match case pattern
- ➔ operador ternário

operadores relacionais

comparam dois valores e **sempre** retorna um booleano (True ou False), são como perguntas que serão respondidas com sim ou não.

== igual a

!= diferente de

> maior que

< menor que

>= maior ou igual a

<= menor ou igual a

*exemplos no arquivo *operadores_relacionais.py*

operadores lógicos

verificam a condição **combinando** respostas, **sempre** retorna True ou False.

and (e): retorna True se todas as condições devem ser True.

or (ou): retorna True se pelo menos uma condição deve ser True.

not (não): inverte o valor, retorna Falso se o valor for True.

guia bem completo: https://www.geeksforgeeks.org/python/python-logical-operators/

*exemplos no arquivo *operadores_logicos.py*

condicionais e pra que servem

condicionais permitem que o código se **ramifique**, criando caminhos que permitem nosso código executar sequencialmente porém ignorando **blocos** inteiros de código com base em condições.

existem **três blocos** condicionais principais em python:

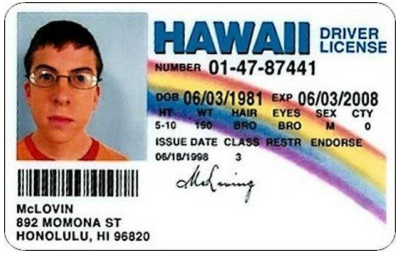
- **if :** (se) decisão inicial, se a condição ocorrer o bloco abaixo é executado.
- **elif :** (senão se) decisão testada caso a anterior não seja atingida, se repete até que todas sem verificadas ou uma delas seja atingida
- **else :** (senão) é o caminho alternativo padrão

exemplo 1

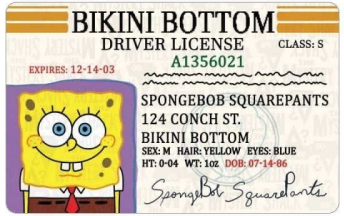
```python
idade = int(input("digite idade: "))

print("verificando se a pessoa é maior de idade... (essa mensagem sempre é exibida)")

if idade >= 18:
    print("voce é maior de idade")
else:
    print("voce é menor de idade")
```




```
digite idade: 24
verificando se a pessoa é maior de idade... (essa mensagem sempre é exibida)
voce é maior de idade
```



exemplo 2

```python
number = float(input("insira um numero: "))

if number > 0:
    print("número positivo")
elif number < 0:
    print("número negativo")
else:
    print("zero")
```

```
insira um numero: -999999997432
número negativo
```

exemplo 3

```python
login = input('insira seu login: ')
senha = input('insira sua senha: ')

LOGIN_USUARIO = 'admin'
SENHA_USUARIO = "2345678"

if (not login or not senha):
    print("voce deve inserir login e senha")
elif (login == LOGIN_USUARIO and senha == SENHA_USUARIO):
    print("acesso permitido")
else:
    print("acesso negado")
```

exemplo 4

```python
saldo_bancario = 1000

valor_saque = float(input("quanto quer sacar: "))

if (valor_saque <= saldo_bancario and valor_saque > 0):
    saldo_bancario -= valor_saque
    print(f"saque realizado com sucesso, valor atual em conta {saldo_bancario}")
else:
    print("saldo insuficiente para saque")
```

```
quanto quer sacar: 1001
saldo insuficiente para saque
```

exemplo 5

```python
fortuna = float(input('qual seu valor acumulado?: '))

if fortuna >= 100000:
    print("VIP")
elif fortuna >= 500000:
    print("premium")
elif fortuna >= 300000:
    print("deluxe")
elif fortuna >= 100000:
    print("intermediario")
elif fortuna >= 50000:
    print("iniciante")
else:
    print("não classificado")
```

```
qual seu valor acumulado?: 1200
não classificado
```

exemplo 6

```python
nota= int(input("insira a nota: "))

if nota>= 90:
    print("excelente nota, bom trabalho!")

if nota>= 60:
    print("você está aprovado")

if nota< 60:
    print("você não passou na prova")
```

```
insira a nota: 80
você está aprovado
```

```
insira a nota: 93
excelente nota, bom trabalho!
você está aprovado
```

nota: 95

nota: 25

exemplo 7

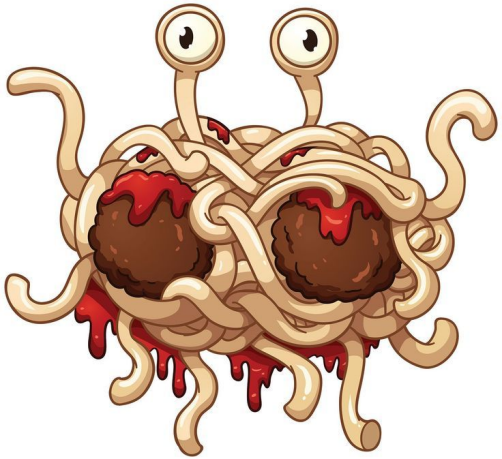
deve-se sempre **evitar ao máximo** o aninhamento de if's, isso torna o código pouco legível e a lógica confusa, gerando o que chamamos de **código espaguete.**

```python
idade = int(input("digite a idade: "))

print("\n** verificacao de idade para chat de voz no roblox... **\n")
if idade >= 14:
    print("idade verificada ✅")
    has_id = input("voce tem rg (sim / não): ")

    if has_id == "sim":
        print("rg verificado ✅")
        is_member = input("voce é membro (sim / não): ")

        if is_member == "sim":
            print("acesso concedido, pode usar o chat de voz 🎉")
        else:
            print("apenas membros tem acesso ao chat de voz")
    else:
        print("rg necessario")
else:
    print("voce precisa ter no minimo 14 anos para usar o chat de voz")
```



erros comuns com condicionais: inicio de bloco

```
emplo5_condicional.py > ...
fortuna = float(inpu
Expected ":" Pylance
View Problem (Alt+F8)   Quick Fix... (Ctrl+.)   Fix (Ctrl+I)

if fortuna >= 100000
    print("VIP")
elif fortuna >= 500000:
    print("premium")
```

a falta de “ : ” após abrir um bloco condicional resulta em erro de sintaxe e falha do código, a indentação também é muito importante pois delimita o bloco. *facilmente identificado pela ide.*

comparações de string, assim como tudo em python, são case sensitive, isso pode influenciar na execução do seu código. é sempre uma boa ideia trabalhar com caracteres sempre em maiúsculo ou minúsculo, e converter todo dado recebido do usuário para seguir o padrão.

```
filme = "The Goodfellas"
filme_input_usuario = "the Goodfellas"

filme == filme_input_usuario  # False
filme == filme_input_usuario.lower() # True
```

erros comuns com condicionais: sinais

atribuição é diferente de comparação!

= representa o operador aritmético de atribuição

== representa o operador relacional de comparação *igual a*

```
fruta = "banana 🍌"
print(fruta)

print(fruta == "banana 🍌")
```

```
banana 🍌
True
```

erros comuns com condicionais: ordem de verificação

no exemplo 1 temos um sistema de verificação de idade simples, ele aceita inclusive valores que não são compatíveis, como por exemplo números negativos ou valores extremamente altos. se quisermos adicionar novas condições baseadas nisso podemos fazer algo parecido com:

```
idade = int(input("digite idade: "))

print("verificando se a pessoa é maior de idade... (essa mensagem sempre é exibida)")

if idade >= 18:
    print("voce é maior de idade")
elif idade < 0:
    print("voce ainda não nasceu!")
elif idade > 120:
    print("voce me parece velho demais...")
else:
    print("voce é menor de idade")
```

idades menores que 0 devem receber uma resposta diferente, assim como valores acima de 120

erros comuns com condicionais: ordem de verificação

o resultado não é o esperado, mesmo com a idade inserida sendo muito maior que o limite estabelecido, ainda estamos no primeiro bloco if.

por que isso acontece?

```
digite idade: 500
verificando se a pessoa é maior de idade... (essa mensagem sempre é exibida)
voce é maior de idade
```

```
digite idade: 121
verificando se a pessoa é maior de idade... (essa mensagem sempre é exibida)
voce é maior de idade
```

erros comuns com condicionais: ordem de verificação

o problema é a **ordem de verificação das condições**, diferente do exercício 6 onde apenas **if**'s foram usados, o que faz com que todas as verificações sejam executadas, validamos a idade no exercício 1 usando **elif**, quando uma das condições for verdadeira, as demais **nunca serão verificadas**, pois python busca pela **primeira verdade** na ordem sequencial.

sempre que houver um **caso específico** que pode cumprir mais de uma condicional, devemos nos preocupar com a **ordem de verificação** ou adicionar **condições mais restritivas**.

erros comuns com condicionais: ordem de verificação

alterando a ordem de verificação podemos resolver o problema dessa forma:

```python
idade = int(input("digite idade: "))

print("verificando se a pessoa é maior de idade... (essa mensagem sempre é exibida)")

if idade > 120:
    print("voce me parece velho demais...")
elif idade >= 18:
    print("voce é maior de idade")
elif idade < 0:
    print("voce ainda não nasceu!")
else:
    print("voce é menor de idade")
```

código no arquivo *exemplo1_condicionalv2.py*

```
digite idade: 500
verificando se a pessoa é maior de idade... (essa mensagem sempre é exibida)
voce me parece velho demais...
```

```
digite idade: 121
verificando se a pessoa é maior de idade... (essa mensagem sempre é exibida)
voce me parece velho demais...
```

montando a verificação em ordem do mais específico, nesse caso da maior idade para a menor.

erros comuns com condicionais: ordem de verificação

outra alternativa é criar condições mais restritivas:

```python
idade = int(input("digite idade: "))

print("verificando se a pessoa é maior de idade... (essa mensagem sempre é exibida)")


if idade >= 18 and idade < 120:
    print("voce é maior de idade")
elif idade < 0:
    print("voce ainda não nasceu!")
elif idade > 120:
    print("voce me parece velho demais...")
else:
    print("voce é menor de idade")
```

```
digite idade: 500
verificando se a pessoa é maior de idade... (essa mensagem sempre é exibida)
voce me parece velho demais...
```

```
digite idade: 121
verificando se a pessoa é maior de idade... (essa mensagem sempre é exibida)
voce me parece velho demais...
```

código no arquivo
exemplo1_condicionalv3.py

match case pattern

se parece com uma painel de escolhas, sempre que um dos **casos** der **match** a lógica será executada.

o match case é mais limpo e legível, e funciona quando as comparações são feitas com valores e não condições.

```python
day = input("insira o dia da semana: ")

match day:
    case "segunda":
        print("semana começando")
    case "sexta":
        print("quase fim de semana")
    case "sábado" | "domingo":
        print("fim de semana")
    case _:
        print("dia normal")
```

```python
idade = int(input("qual sua idade: "))

# não é possível usar comparações diretas
match idade:
    case idade > 120:
        print("muito velho")
    case idade >= 18:
        print("adulto")
```

operador ternário

funcionalidade avançada que permite simplificar uma condicional em uma única linha, porém pode dificultar a legibilidade do código.

é especialmente útil para ser utilizado dentro de **fstrings** em um **print()**

```
# exemplo print
clima = "chuva"
print(f"vou levar um {'guarda-chuva' if clima == 'chuva' else 'boné'}.")
```

```
vou levar um guarda-chuva.
```

```
# classificando idade
idade = int(input("digite a idade da pessoa: "))

#condicional
if idade >= 18:
    tipo_de_pessoa = "adulto"
else:
    tipo_de_pessoa = "menor"
print(f"\nessa pessoa é um {tipo_de_pessoa}.")

#ternário
tipo_de_pessoa = "adulto" if idade >= 18 else "menor"
print(f"\nfeito com ternário: essa pessoa é um {tipo_de_pessoa}.")
```

código no arquivo
operador_ternariopy

o que é a verdade (em python)

em python, não apenas valores booleanos (True e False) podem ser usados em condicionais. quase tudo tem valor lógico.

considerados **False** (além das condicionais lógicas e relacionais):

- número zero tanto em ***int 0*** quanto em ***float 0.0***
- **strings vazias**, lembre-se que espaços são considerados caracteres.
- **listas, tuplas, dicionários, sets vazios.**
- o valor ***None***

considerados **True** (além das condicionais lógicas e relacionais):

- qualquer número **int** ou **float diferente de zero**
- **strings com ao menos um caractere** incluindo espaços.
- **listas, tuplas, dicionários** com ao menos um elemento.

exercicios

1 calculadora com inputs 🟢

solicite ao usuário **dois números**, após isso **solicite um dos operadores**:

+ soma

- subtracao

/ divisao

* multiplicacao

** potenciacao

baseado na entrada **execute a operação** e **exiba o valor**.

verificador de idade 🟢

solicite ao usuário sua idade e classifique em:

- bebê - entre 0 e 2 anos
- adulto - entre 18 e 65
- criança - entre 3 e 13
- idoso - entre 65 e 120
- erro - abaixo de 0
- adolescente - entre 13 e 18
- múmia - a partir de 120

corrigindo código: continue? 🟢

- o código deve exibir a mensagem “continuando...” quando o input do usuário for “sim”.
-

```
answer = input("continuar? (sim / não): ")

if answer == sim:
    print("continuando...")
else:
    print("parando...")
```

clima no rio de janeiro 🟢

pergunte ao usuário três questões a ser respondidas com “sim” ou “não”:

- está chovendo?
- está sol?
- ouve tiros?

baseado nisso defina se o usuário **deve ou não** sair com **guarda chuva**.

se estiver chovendo voce deve levar, exceto quando ouve tiros.

em dias de sol não há necessidade.

em dias de apenas tiros também não

par ou ímpar 🟢

solicite um **número inteiro** e informe se ele é **par** ou **ímpar**.

corrigindo código: validando idade básica 🟢

- corrija o código abaixo, ele deve retornar corretamente a classificação de acordo com a idade

```
age = input("digite sua idade:
")

if age ⩾ 18:
    print("adulto")
else:
    print("menor")
```

maior menor ou igual 🟢

solicite dois números e informe se um deles é **maior**, **menor** ou **igual** ao outro.

validando hora e minuto 🟢

solicite **horas** e **minutos** do usuário e verifique se são válidos, **no formato 24h.**

ex:

- 23:45 **válido**
- 25:54 **inválido**

semáforo match-case 🟡

usando match-case pattern, replique o funcionamento de um semáforo exibindo uma mensagem sobre o que cada cor indica. 🚦

corrigindo código: nota do aluno🟡

- o código abaixo deve diferenciar alunos que passaram com nota acima a média (60) dos demais informando uma mensagem especial, corrija o problema que faz com que isso não ocorra.

```
score = int(input("nota do aluno: "))

if score ⩾ 60:
    print("passou na média")
elif score ⩾ 90:
    print("passou com excelente nota")
else:
    print("reprovado")
```

enviando pacotes 🟡

crie uma lógica para classificar pacotes que variam de peso entre

- 1KG e 30KG
- 10cm e 1m
- expresso ou normal

classifique em no mínimo três categorias e faça uso de ao menos duas informações do pacote.

o código final deve solicitar as três informações e retornar a classificação.

(você pode adicionar mais informações se quiser :p)

calculadora imc 🟡

solicite ao usuário seu **peso** e **altura**. realize o cálculo do imc, e exiba o valor encontrado e a classificação.

abaixo de 18.5: "Abaixo do peso"

entre 18.5 e 25: "Peso ideal"

entre 25 e 30: "Sobrepeso"

acima de 30: "Obesidade"

calculator nota e p3 🟡

solicite a nota do aluno na P1 e P2, verifique se o aluno precisa fazer a P3, ou se está aprovado.

Em caso de P3 solicite a nota P3 e indique o estado do aluno.

considere a média 7.

validando triangulo 🟡

solicite o valor de três retas, verifique se elas podem formar um triângulo (soma de dois lados maior que terceiro) e informe o resultado.

se formarem um triângulo, informe o tipo:

- equilatero: tres iguais
- isosceles: dois iguais
- escaleno: tres diferentes

validar senha 🟡

solicite que o usuário insira seu **nome** e sua **senha**, faça a validação da senha verificando as condições:

- deve ter no minimo 6 caracteres
- no maximo 16 caracteres
- **não pode conter ter espaços**

informa a situação da senha.

eleições presidenciais 🟡

solicite a **idade do usuário** e sua **nacionalidade**, se o usuário tiver **16 anos ou mais** e for da **nacionalidade “Brasileiro”** ele pode prosseguir, do contrário deve se encerrar o programa. em seguida pergunte se o usuário **possui título de eleitor**, verifique e exibe o resultado.

ano bissexto 🔴

solicite do usuário um **ano**, informe se ele é um ano bissexto.

um ano só é bissexto quando podemos dividir ele por 4, mas se esse mesmo ano for divisível por 100 ele não será bissexto, a não ser que possamos dividi-lo por 400! 😵‍💫

calculando area de triangulos 🔴*desafio*🔴

solicite ao usuário que informe qual o tipo de triangulo a ser calculado inserindo o valor seguindo a legenda:

- Equilátero “1”
- Isósceles “2”
- Escaleno “3”
- Retângulo“4”

baseado na entrada, solicite as medidas necessárias, realize e exiba o cálculo.

dica: você pode usar math.sqrt(2) para calcular a raiz quadrada.

adicione *import math* no começo do seu código.

```
import math
print(math.sqrt(16))
4.0
```

pedra papel e tesoura🔴*desafio*🔴

solicite a **jogada** de cada um dos jogadores (pedra, papel ou tesoura), lembre-se de usar **lower()** para **padronizar os dados**.

use **condicionais** para criar a **lógica que define o vencedor** de acordo com as regras clássicas do jogo.

informe ao final o **vencedor** ou um **empate**.